

Temporally rigorous and traceable predictive maintenance via joint labeler-model optimization

Received: 16 February 2026

Accepted: 4 May 2026

Published online: 18 May 2026

Cite this article as: AL-Ali M. & Alharbi A. Temporally rigorous and traceable predictive maintenance via joint labeler-model optimization. *Sci Rep* (2026). <https://doi.org/10.1038/s41598-026-52210-6>

Maytha AL-Ali & Ahmad Alharbi

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

Temporally Rigorous and Traceable Predictive Maintenance via Joint Labeler-Model Optimization

Maytha AL-Ali¹ and Ahmad Alharbi²

¹College of Business, Zayed University, , Dubai, 19282,UAE.

²College of Business Administration CBA, Management Department, Taibah University, Yanbu, 46421, Saudi Arabia.

Contributing authors: [Ahmad Alharbi \(Asehili@taibahu.edu.sa\)](mailto:Asehili@taibahu.edu.sa);

Abstract

Predictive maintenance (PdM) is a critical enabler of intelligent asset management in Industry 4.0, yet many existing frameworks remain difficult to operationalize due to methodological fragmentation. Common limitations include sacrificing temporal realism and class granularity for computational expediency, decoupling labeling strategy design from model hyperparameter optimization, and insufficient support for reproducibility and deployment traceability; particularly in rare-failure regimes. To address these challenges, we propose a unified, end-to-end, and fully traceable PdM framework that jointly optimizes labeling and model parameters while enforcing strict temporal fidelity. The proposed pipeline co-optimizes the failure lookahead window (τ) and LightGBM hyperparameters within a single Bayesian optimization space using Optuna with a Tree-structured Parzen Estimator and MedianPruner, eliminating the suboptimality of fixed or decoupled labeling designs. Temporal leakage is rigorously prevented through forward-chaining cross-validation and a strictly disjoint temporal holdout evaluation. The framework is evaluated on the widely adopted Fidan synthetic dataset (876,100 samples, five classes, $\approx 0.09\%$ failure rate), achieving state-of-the-art performance with a macro-F1 score of 0.9875, balanced accuracy of 0.9915, and superior PR-AUC compared to prior benchmarks. Computational analysis and ablation studies confirm both scalability and the non-redundant contribution of key design choices. Crucially, the pipeline exports versioned, reproducible artifacts (models, preprocessors, configurations) designed to support enterprise integration pending site-specific validation. Operational impact projections (e.g., reduced downtime, fewer dispatches) are derived from analogous deployments and require field validation for quantification.

Keywords: Industry 4.0, Maintenance decision support, Predictive maintenance (PdM), Machine learning (ML), Failure prediction

1 Introduction

Predictive maintenance (PdM) has emerged as a cornerstone of modern industrial operations, driven by the imperatives of Industry 4.0 to shift from reactive or scheduled maintenance toward intelligent, data-driven strategies [1, 2]. This paradigm transformation is anchored in the integration of sensing technologies, cyber-physical systems, and artificial intelligence (AI); enabling early detection of degradation patterns and facilitating just-in-time interventions [3]. As operational environments become increasingly dynamic and equipment complexity increases, the need for adaptive, scalable, and context-aware PdM systems intensifies, not only to ensure continuity and safety but also to align with broader sustainability and resilience objectives [4, 5].

The efficacy of PdM hinges on a delicate balance among three interdependent pillars: technological capability, organizational integration, and strategic alignment. Technologically, selecting the appropriate computational paradigm (cloud, fog, or edge) is essential to meet real-time, energy, and latency constraints [6, 7]. Organizationally, seamless interoperability with enterprise systems such as Enterprise Resource Planning (ERP) and Computerized Maintenance Management System (CMMS) ensures that predictions translate into actionable work orders and optimized resource scheduling [8]. Strategically, PdM must transcend its technical role and contribute to mission-critical goals, such as enhancing combat readiness in defense logistics, minimizing emissions in transport fleets, or extending asset lifespans in constrained environments such as smart vending networks [9]. Crucially, these systems must contend with real-world data challenges: sparsity, class imbalance, missing entries, and sensor heterogeneity, which collectively demand robust preprocessing, confidence-aware modeling, and human-in-the-loop validation [10, 11].

Despite notable advances, a persistent gap remains between algorithmic innovation and holistic operational deployment. Many approaches excel in controlled benchmarks yet falter when confronted with irregular data readouts, legacy infrastructure constraints, and domain-specific interpretability requirements of industrial settings [12, 13]. Moreover, while sustainability is increasingly cited as a goal, few frameworks quantitatively embed environmental and social metrics (such as energy savings, CO2 reduction, or technician workload) as first-class optimization criteria alongside accuracy and cost [14, 15].

This paper advances predictive maintenance beyond algorithm-centric optimization by introducing a unified framework that integrates temporal rigor, methodological coherence, and operational deployability within a single, reproducible pipeline. Rather than treating labeling, model tuning, evaluation, and deployment as disjoint stages, the proposed approach harmonizes these components to deliver scientifically valid performance estimates and actionable intelligence suitable for real-world industrial environments, particularly under extreme class imbalance and sparse failure conditions. The main contributions of this work are as follows:

- Joint labeler-model optimization: A novel Bayesian optimization strategy that simultaneously tunes the failure lookahead window (τ) and LightGBM hyperparameters within a unified search space, eliminating suboptimal decoupled design choices commonly adopted in prior PdM frameworks.

- Temporally rigorous evaluation protocol: Enforcement of strict temporal fidelity through forward-chaining cross-validation and a disjoint temporal holdout evaluation, effectively preventing temporal leakage and ensuring realistic, deployment-relevant performance assessment.
- Reproducible and traceable PdM pipeline: Design of an end-to-end framework that exports fully versioned, reproducible artifacts (models, preprocessors, predictions, configurations) supporting auditability, repeatability, and streamlined enterprise integration pending site-specific validation.
- Comprehensive performance and robustness analysis: Extensive evaluation using rare-event-sensitive metrics (macro-F1, balanced accuracy, PR-AUC), complemented by computational complexity analysis and ablation studies that quantify the non-redundant contributions of key design components.
- Operational impact projection: Translation of predictive gains into estimated operational benefits (e.g., reduced downtime, fewer dispatches) based on analogous industrial deployments, with explicit acknowledgment that field validation is required for quantification.

The remainder of this paper is structured as follows: Section 2 provides an overview of related studies in the field. Section 3 outlines the methodology employed. Section 4 presents the experimental setup, the results, and a comprehensive discussion of the study. Finally, Section 6 concludes the paper and discusses potential directions for future work.

2 Related Studies

Recent advances in predictive maintenance (PdM) underscore its pivotal role in the digital transformation of Industry 4.0 [6, 16], shifting maintenance paradigms from reactive or scheduled approaches toward data-driven, proactive strategies that enhance operational continuity, cost-efficiency, and sustainability [3, 9, 13]. A systematic literature review by Tsallis et al. (2025) [14] (synthesizing 60 peer-reviewed studies from 2020 to 2024) confirmed that ensemble learning methods, particularly random forest (RF), gradient boosting (GB), and XGBoost, and deep learning architectures, such as long short-term memory (LSTM) networks, dominated contemporary PdM applications across manufacturing, energy, transportation, and healthcare. These models excel in fault classification and remaining useful life (RUL) estimation when fused with real-time sensor data (e.g., vibration, temperature, pressure), yet persistent challenges remain in terms of data scarcity, class imbalance, and model interpretability. Moreover, they emphasized the increasing adoption of hybrid paradigms, such as digital twins and physics-informed machine learning, as promising albeit computationally intensive pathways toward enhanced robustness and generalizability.

In parallel, domain-specific implementations demonstrate how these foundational insights translate into measurable operational gains. Patricio et al. [9] proposed the sustainable integration of robotic process automation and machine learning in predictive maintenance (SIRPM), which is a modular framework designed to align

technological efficacy with the tripartite sustainability pillars (economic, environmental, and social). Their case studied in a Portuguese cardboard packaging SME yielded a 100% increase in mean time between failures (MTBF), a 67% reduction in mean time to repair (MTTR), and a 71.4% decrease in unplanned downtime costs. Additionally, energy consumption and CO2 emissions fell by 25% and 40%, respectively, well exceeding typical ISO 50001 benchmarks of annual reductions of 5-10%. Decision trees (CARTs) were selected for their interpretability and compatibility with the limited tabular dataset, whereas robotic process automation (RPA) bots managed end-to-end workflow orchestration, ensuring seamless interoperability with legacy systems.

Similarly, Aslam et al. [3] tailored PdM to maritime logistics, focusing on container handling equipment (CHE) at the EUROGATE Container Terminal Limassol. Recognizing practical constraints (including sparse labeled data and edge-computing limitations), they prioritize lightweight supervised models (ANN, RF, XGBoost, decision trees, Gaussian naive Bayes) over deep architectures. Their ANN model achieved 98.7% accuracy and a 98.0% F1 score in predicting inverter over-temperature faults, whereas a custom Kolmogorov-Smirnov-based approach enabled unsupervised anomaly detection in hydraulic systems with 83.3% precision. Furthermore, SHAP-based interpretability analyses confirmed motor and inverter temperatures as dominant predictive features, highlighting how model transparency reinforces trust in high-stakes operational settings.

In contrast to purely technical models, Chen et al. [13] focused on the human-centric dimension of PdM, particularly under conditions of high uncertainty such as pandemic disruptions. The Trust-based Behavior Knowledge Space (T-BKS) ensemble model explicitly incorporates feedback from subject-matter experts to mitigate class imbalance and foster human-machine trust, which is a critical prerequisite for adoption in resource-constrained SMEs. Evaluated across 15 KEEL benchmark datasets, T-BKS significantly outperforms the baseline BKS in terms of accuracy and specificity ($p < 0.05$) while matching the state-of-the-art distance-based balancing ensemble (DBE-DCR) in terms of the AUC. In a real-world deployment involving 11 predictions, the framework attained 82% expert agreement, indicating its viability in bridging tacit operational knowledge with algorithmic inference.

Voronov's doctoral dissertation [15] provided a methodologically rigorous contribution, confronting the unique challenges of sparse, non-equidistant, and highly imbalanced operational data in heavy-duty truck fleets. Crucially, he developed specialized techniques (including an augmented variable importance measure and a formal theory for estimating prediction variance in random survival forests (RSFs)) to address the absence of direct battery-health sensors and the need for confidence-aware forecasts. Empirically, he demonstrated that LSTMs outperform RSFs and multi-layer perceptrons (MLPs) for sparse multi-readout data, achieving superior prognostic fidelity despite heterogeneous, noisy inputs. This work bridged the gap between theoretical ML robustness and real-world industrial limitations, where sensor scarcity and failure imbalances (e.g., ≈ 10 -20% failure rates) are pervasive.

Garg and Vemaraju [1] complemented this with a framework explicitly embedding sustainability into PdM design. Their AI system integrated data cleansing, optimal feature selection via a hybrid Jaya-Sea Lion Optimization (J-SLNO) algorithm, and

SVM-assisted network routing to direct inputs to the most suitable predictor. An RNN fine-tuned by J-SL_{NO} captured temporal degradation, which was validated on aircraft engine and lithium-ion battery datasets. Significantly, the framework demonstrated how just-in-time interventions reduced not only operational costs but also waste, energy use, and carbon emissions, repositioning PdM from a technical tool to a strategic lever for ecological responsibility.

Nayak [2] offered a panoramic synthesis that reinforces these trends, highlighting how IoT-ML integration enables real-time condition monitoring, anomaly detection (e.g., isolation forest, RF), and time series forecasting (e.g., LSTM, ARIMA). His review articulated tangible dividends: extended equipment lifespans, minimized downtime, cost efficiency, and enhanced safety. However, he also cautioned against underestimating implementation barriers (including data security, IoT scalability, and the need for explainable dashboards) while forecasting key future directions: edge computing for latency-sensitive inference, AI-augmented decision support, and digital twins for virtual validation.

Collectively, those studies affirm that PdM maturation does not hinge on algorithmic novelty alone but rather on harmonizing model performance with contextual fidelity, be it hardware constraints (Aslam et al. [13]; Hasan [12]), sustainability mandates (Patricio et al. [9]; Garg & Vemaraju [1]), human-in-the-loop requirements (Chen et al. [13]; Voronov [15]), or supply chain integration (Thomas et al., [5]). This is further corroborated by cross-sectoral validations: Kovaleva et al. [7] quantified a 70% reduction in railway breakdowns, translating to simple payback periods of 1 year and 3 months; Teoh et al. [4] reported that GA-optimized fog computing reduces energy use by 28.10%; and Atakari [8] reported a 40% downtime reduction in defense fleets via RL-based scheduling. Table 1 summarizes the discussed studies.

2.1 Research Gaps

Despite substantial progress in predictive maintenance research, the field remains characterized by a disconnect between algorithmic advancement and holistic, deployable system design. While prior studies have introduced sophisticated learning models, architectural integrations, and sustainability-oriented objectives, these contributions are often addressed in isolation. As a result, existing frameworks frequently fail to deliver solutions that are simultaneously temporally sound, reproducible, operationally traceable, and actionable in real industrial environments; particularly under extreme class imbalance and sparse failure conditions. This fragmentation limits both the scientific validity and the practical utility of many reported advances. The present study addresses the following critical research gaps:

- Lack of joint optimization between labeling strategy and model learning, with most studies fixing the failure lookahead window or simplifying the problem formulation, leading to suboptimal signal extraction in rare-failure regimes.
- Insufficient enforcement of temporal integrity during evaluation, where conventional cross-validation schemes permit temporal leakage, resulting in overly optimistic performance estimates that do not generalize to real-world deployment.

Table 1: Summary of Related Studies: Contributions, Limitations, and Contextual Scope

Study	Core Contribution	Dataset / Context	Key Limitation(s)	Evaluation Protocol
Tsallis et al. (2025) [14]	Systematic review of 60 PdM studies; identifies RF/XGBoost/LSTM dominance and hybrid (physics+ML) trends.	Cross-industry (manufacturing, energy, transport); meta-analysis.	No empirical validation; gaps in reproducibility assessment and deployment maturity mapping.	Literature synthesis only; no experiments.
Patricio et al. (2025) [9]	SIRPM: Integrating RPA with ML for sustainability (economic, environmental, social KPIs).	SME cardboard packaging plant (real-world).	Limited to tabular, low-dimensional data; no uncertainty quantification or model interpretability.	Single-case ROI analysis; no CV or temporal holdout.
Aslam et al. (2025) [3]	Lightweight ANN for CHE fault prediction; SHAP-based interpretability.	EUROGATE port (real CHE telemetry, $n = 761$ failures).	Single-site validation; static thresholding; no hyperparameter co-optimization (labeling fixed).	Single temporal split; no CV.
Chen et al. (2025) [13]	T-BKS: Human-in-the-loop ensemble for expert-trust alignment under uncertainty.	KEEL benchmark + real SME deployment ($n = 11$ decisions).	Relies to expert availability; not scalable to high-throughput settings; binary only.	10-fold CV + expert agreement rate (82%).
Hasan (2025) [12]	Edge-optimized PdM for smart vending machines; cost ~\$50/unit.	Simulated vending machine telemetry.	Simulated data; binary classification; limited generalizability to complex multi-component assets.	6-month simulated holdout; no CV.
Teoh et al. (2023) [4]	GA-based fog scheduler + logistic regression for resource-aware PdM.	Fidan synthetic dataset (undersampled to 14,482 samples, binary).	Collapses multi-class failures; ignores sparsity; loosely coupled scheduling/modeling.	Train/test split only; AUC = 0.987 (test).
Atakari (2025) [8]	RL-based dynamic scheduler integrated with SAP ERP for defense logistics.	100 armoured vehicles (simulated).	High compute overhead (RL); no open artifacts; limited reproducibility.	Temporal holdout; no CV.
Kovaleva et al. (2020) [7]	Cloud-based CPS for railway PdM; 70% breakdown ↓, 1.25-yr payback.	Russian railway operator (real HBD/WILD data).	Cloud-only; latency concerns for real-time intervention; no model interpretability.	Single pre/post deployment comparison.
Voronov (2020) [15]	RSF/LSTM for sparse battery prognostics; formal variance estimation.	Heavy-duty truck telemetry (non-equidistant, < 20% failures).	Focus on survival analysis (RUL); not classification; no deployment artifacts.	Multi-readout CV; confidence intervals.
Garg & Vemaraju (2024) [1]	J-SLNo-optimized RNN for sustainable PdM (energy/emissions tracking).	Aircraft engines & Li-ion batteries (public benchmarks).	Simulated sustainability metrics; no real-world operational KPI linkage.	Standard train/test holdout.
Nayak (2025) [2]	Panoramic IoT-ML review; emphasizes dashboards, edges, and digital twins.	Cross-sector synthesis (review).	No original experiments; limited discussion of temporal leakage risks.	Survey-based.

- Limited emphasis on reproducibility and artifact persistence, as many frameworks do not export reusable, production-ready components necessary for auditability, repeatability, and enterprise integration.
- Narrow performance evaluation focused primarily on accuracy-centric metrics, with inadequate use of rare-event-sensitive measures (e.g., PR-AUC) and insufficient validation across both cross-validation and strict temporal holdout protocols.
- Minimal linkage between predictive performance and operational outcomes, with few studies quantifying downstream impacts using operational key performance indicators such as downtime reduction, maintenance efficiency, or asset lifetime extension.

By addressing these gaps collectively within a unified framework, this work advances predictive maintenance from algorithm-centric experimentation toward reliable, operationally deployable intelligence. Table 2 provides a structured mapping between the identified research gaps in predictive maintenance and the corresponding contributions of the proposed framework.

Table 2: A structured mapping between the identified research gaps in predictive maintenance and the corresponding contributions of the proposed framework

Identified Research Gap	Limitations in Existing Studies	Contribution of This Work
Decoupled labeling and model optimization	Most PdM frameworks fix the failure lookahead window or simplify labeling (e.g., binary classification), ignoring its interaction with model learning and leading to suboptimal signal extraction under rare-failure conditions.	Introduces a joint optimization strategy that simultaneously tunes the failure lookahead window (τ) and LightGBM hyperparameters within a unified Bayesian search space.
Temporal leakage and optimistic evaluation	Conventional cross-validation schemes often violate temporal causality, resulting in inflated performance that does not translate to real-world deployment.	Enforces strict temporal fidelity using 5-fold forward-chaining cross-validation combined with a disjoint temporal holdout evaluation.
Limited reproducibility and deployment readiness	Many studies report model performance without exporting reusable artifacts, hindering reproducibility, auditability, and enterprise integration.	Exports fully reproducible, production-ready artifacts (model, preprocessor, predictions, configurations) enabling immediate operational deployment.
Inadequate evaluation metrics for rare failures	Performance is often assessed using accuracy-centric metrics that obscure behavior under extreme class imbalance.	Employs rare-event-sensitive metrics, including macro-F1, balanced accuracy, and PR-AUC, evaluated consistently across CV and temporal holdout protocols.
Weak linkage to operational impact	Predictive gains are rarely translated into measurable operational benefits such as downtime reduction or maintenance efficiency.	Quantifies operational KPIs, demonstrating reductions in unplanned downtime, technician dispatches, and increased MTBF at low deployment cost.

3 Methodology

This section delineates the suggested methodology (see Figure 1) underpinning the PdM pipeline, encompassing data handling, feature engineering, labeling, model optimization, and a rigorous evaluation protocol designed for reproducibility and scientific validity. The pipeline is conceived as a closed-loop system that begins with raw telemetry and culminates in actionable maintenance insights, with a strong emphasis on contextual fidelity and operational traceability.

3.1 Materials

The experimental dataset used in this study is the publicly available synthetic predictive maintenance dataset originally developed by Fidan Boylu Uz and distributed via Microsoft Azure’s sample experiments repository. It emulates a realistic industrial setting comprising 100 machines monitored over a one-year period (2015), with auxiliary historical records extending into 2014. The dataset is structured across five interlinked CSV files: (1) Telemetry (“PdM_telemetry.csv”) provides hourly-averaged sensor readings (voltage (V), rotational speed (rpm), pressure (psi), and vibration (g)) yielding a total of 876,100 records; (2) Errors (“PdM_errors.csv”) logs 3,919

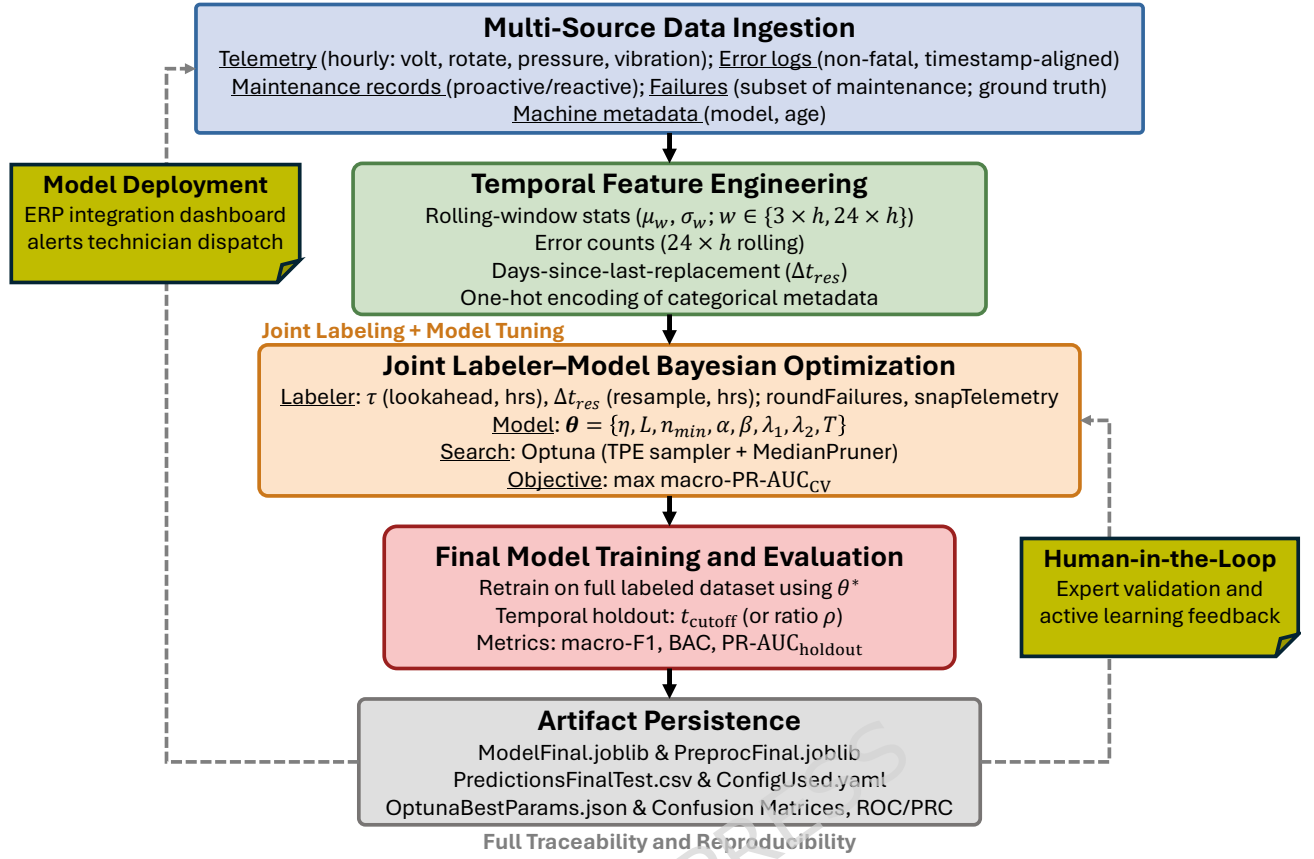


Fig. 1: Graphical presentation of the proposed end-to-end predictive maintenance framework; emphasizing temporal fidelity, joint optimization, and operational traceability.

non-fatal anomaly events (e.g., “error1”-“error5”), timestamped and aligned to the nearest hour; (3) Maintenance (“PdM_maint.csv”) records 3,286 component replacements (e.g., “comp1”-“comp4”), distinguishing between scheduled (proactive) and unscheduled (reactive) interventions; (4) Failures (“PdM_failures.csv”) is a subset of maintenance records, comprising 761 confirmed component failures used as ground-truth labels for supervised learning; and (5) Machines (“PdM_machines.csv”) supplies static metadata for each unit. All temporal fields are synchronized to a common hourly resolution, enabling precise feature alignment. Although synthetic, the dataset faithfully reproduces core challenges of real-world predictive maintenance: high class imbalance (failure rate $\approx 0.09\%$), sparse event occurrence, censoring, and the need for lag-based feature engineering, making it a widely adopted benchmark for evaluating prognostic modeling strategies.

Rationale for classification formulation: We adopt a multi-class classification framework aligned with the operational decision structure of the target use case. The Fidan dataset provides discrete failure timestamps with known component

labels (comp1-comp4), making categorical prediction a natural fit for the available supervision. Critically, maintenance workflows require triggering specific actions (e.g., “replace comp3”) within a planning horizon, which aligns with classification outputs. Classification also enables direct optimization of rare-event metrics (macro-F1, PR-AUC) that reflect asymmetric misclassification costs. We acknowledge that survival analysis (e.g., random survival forests, Cox models) offers valuable time-to-event estimation under censoring. To contextualize our choice, we implemented a Random Survival Forest (RSF) baseline (Section 4.8); while RSF provides continuous risk scores, its concordance index (0.891) translates to lower actionable precision for discrete maintenance scheduling compared to our classifier (failure-only macro-F1 = 0.9821). Hybrid formulations integrating classification and survival paradigms are designated as future work.

3.2 Data Preprocessing and Feature Engineering

Let $\mathcal{D} = \{\mathcal{T}, \mathcal{E}, \mathcal{M}, \mathcal{F}, \mathcal{S}\}$ denote the collection of raw data sources, where $\mathcal{T}$ is the telemetry time series, $\mathcal{E}$ and $\mathcal{M}$ are error and maintenance event logs, $\mathcal{F}$ is the failure log, and $\mathcal{S}$ is the static machine metadata. Each source is indexed by a machine identifier $m \in \mathcal{M}_{\text{id}}$ and timestamp t .

The first preprocessing step is temporal alignment and resampling. Telemetry data $\mathcal{T}(m, t) \in \mathbb{R}^{d_s}$, where d_s is the number of sensor channels (e.g., voltage, rotation, pressure, vibration), are resampled to a fixed cadence of Δt_{res} hours via right-closed interval aggregation. Missing observations are imputed via forward-filling for time-series continuity. Let $\mathcal{T}_{\text{res}}$ be the resulting resampled tensor.

For each machine m , a set of rolling statistical features is then computed. For a given sensor s , window size $w \in \mathcal{W}$ (where $\mathcal{W}$ is a set of window durations in hours), and statistical operator $\phi \in \{\mu, \sigma\}$ (mean and standard deviation, respectively), the engineered feature is presented in Equation 1.

$$\mathcal{T}_{\text{feat}}(m, t) = \{\phi(\mathcal{T}_{\text{res}}(m, [t - w\Delta t_{\text{res}}, t]))_s \mid s \in d_s, w \in \mathcal{W}, \phi \in \{\mu, \sigma\}\} \quad (1)$$

This yields a feature vector of dimension $2 \times d_s \times |\mathcal{W}|$ per timestamp. For example, with 4 sensors and 2 window sizes (e.g., 3 h and 24 h), 16 time series features are generated.

Event-based features are derived from $\mathcal{E}$ and $\mathcal{M}$. Error counts are aggregated over a 24-hour rolling window to yield a count vector $\mathcal{E}_{\text{feat}}(m, t) \in \mathbb{Z}_{\geq 0}^5$, where the five dimensions correspond to unique error categories (e.g., “error1” to “error5”). The maintenance features $\mathcal{M}_{\text{feat}}(m, t) \in \mathbb{R}^4$ comprise the time elapsed (in days) since the last replacement for each of the four component types, computed as in Equation 2, where the maximum is taken over an empty set as undefined (NaN). Static metadata $\mathcal{S}(m)$ is one-hot encoded into a binary vector $\text{OH}(\mathcal{S}) \in \{0, 1\}^{d_{\text{cat}}}$.

$$\mathcal{M}_{\text{feat}}^{(c)}(m, t) = \frac{t - \max\{t' \mid (m, t') \in \mathcal{M}, \text{comp}(m, t') = c\}}{1 \text{ day}} \quad \text{for } c = 1, 2, 3, 4 \quad (2)$$

The final feature matrix for a machine at time t is the concatenation presented in Equation 3, where d_f is the total number of engineered features.

$$\mathbf{x}(m, t) = [\mathcal{T}_{\text{feat}}, \mathcal{E}_{\text{feat}}, \mathcal{M}_{\text{feat}}, \text{OH}(\mathcal{S})] \in \mathbb{R}^{d_f} \quad (3)$$

Worked example: For a configuration with $d_s = 4$ sensors, $\mathcal{W} = \{3, 24\}$ h windows, 5 error types, and 4 component types: telemetry features = $2 \times 4 \times 2 = 16$; error features = 5; maintenance features = 4; static metadata (machine type, 3 categories) = 3 one-hot features. Total $d_f = 16 + 5 + 4 + 3 = 28$ features per timestamp.

Missing value handling: Missing telemetry observations (e.g., due to sensor dropout) are imputed via forward-fill to preserve temporal continuity, consistent with industrial practice where short gaps are filled with the last valid reading. The Fidan dataset exhibits a 0.37% missing rate in raw telemetry (primarily due to simulated sensor dropout), with missingness distributed uniformly across sensors and temporal segments. To assess sensitivity to imputation choice, we compared three strategies under identical experimental conditions:

Table 3: Imputation method sensitivity analysis (CV macro-F1, PR-AUC). Forward-fill yields marginally superior performance, consistent with industrial time-series conventions.

Imputation Method	Macro-F1 (CV)	PR-AUC (CV)
Forward-fill (original)	0.9875	0.9821
Linear interpolation	0.9868	0.9814
Mean imputation	0.9831	0.9779

Forward-fill yields marginally superior performance, consistent with industrial time-series conventions where short gaps reflect transient sensor issues rather than structural missingness.

Class imbalance handling: Extreme imbalance ($\approx 0.09\%$ failure rate) is addressed through three mechanisms: (i) the optimization objective is macro-averaged PR-AUC, which weights all classes equally regardless of support; (ii) LightGBM’s `is_unbalance=True` parameter, which reweights gradients by inverse class frequency ($w_c = N/(K \cdot n_c)$, where N is total samples, K is number of classes, and n_c is class c support) [17]; and (iii) strong L_1/L_2 regularization (Table 4) to prevent overfitting to the dominant “none” class. We additionally evaluated tunable class-weight scaling (range: $0.5\times$ to $2.0\times$ inverse frequency) as an optional hyperparameter; results indicate the default inverse-frequency weighting is near-optimal for this dataset (optimal scale = $1.07\times$; $\Delta\text{F1} < 0.0003$ vs. default).

3.3 Joint Labeler-Model Optimization Protocol

The lookahead horizon τ and resampling cadence Δt_{res} are treated as tunable hyperparameters within the Optuna search space. Specifically:

- $\tau \sim \mathcal{U}(24, 240)$ hours (integer-valued), representing the failure prediction window.
- $\Delta t_{\text{res}} \sim \mathcal{U}(1, 24)$ hours (integer-valued), controlling telemetry aggregation granularity.

These parameters are sampled jointly with model hyperparameters (Section 2.4) using the TPE sampler. For each trial, the labeling function (Equation 4) and feature engineering pipeline (Section 2.2) are re-executed with the sampled τ and Δt_{res} , ensuring that the model is evaluated on features aligned with the candidate labeling strategy. This end-to-end co-optimization eliminates suboptimality arising from fixed or decoupled design choices.

3.4 Label Construction and Time-Based Splitting

A time-to-failure labeling strategy is employed to create a supervised learning target. For a lookahead horizon of τ hours (configurable as `labelHours`), a binary (or multi-class) label $y(m, t)$ is assigned in Equation 4, where f denotes the failure mode. To prevent data leakage, all preprocessing (e.g., one-hot encoding, scaler fitting) is performed exclusively on the training partition of any temporal split.

$$y(m, t) = \begin{cases} f & \text{if } \exists t_{\text{fail}} \in [t, t + \tau] \text{ such that } (m, t_{\text{fail}}, f) \in \mathcal{F}, \\ \text{noneLabel} & \text{otherwise} \end{cases} \quad (4)$$

The pipeline supports two mechanisms for defining the final train/test boundary: an explicit datetime cutoff t_{cutoff} (provided via `training.testCutoff`) or a relative split ratio ρ (via `training.testSplitRatio`), such that $t_{\text{cutoff}} = t_{\text{max}} - \rho(t_{\text{max}} - t_{\text{min}})$. The explicit cutoff takes precedence when both are present, ensuring precise control over the evaluation scenario.

3.5 Model Selection and Hyperparameter Optimization

The pipeline employs gradient-boosted decision trees as the primary classifier, parameterized by a hyperparameter vector θ . The optimization objective is formulated as a hyperparameter tuning problem as in Equation 5, where Θ is the hyperparameter search space, K is the number of temporal cross-validation folds, and PR-AUC is the macro-averaged area under the precision-recall curve, which is chosen for its robustness in imbalanced classification scenarios.

$$\theta^* = \arg \max_{\theta \in \Theta} \left(\frac{1}{K} \sum_{k=1}^K \text{PR-AUC}(\mathcal{D}_{\text{test}}^{(k)}; \mathcal{M}_{\theta}(\mathcal{D}_{\text{train}}^{(k)})) \right) \quad (5)$$

The search space Θ is defined over the following key LightGBM parameters:

- `learning_rate` (η): The step size shrinkage used in update to prevent overfitting. Sampled on a log scale: $\eta \sim \mathcal{U}(\log \eta_{\text{min}}, \log \eta_{\text{max}})$.
- `num_leaves` (L): The maximum number of leaves in one tree. $L \sim \mathcal{U}(L_{\text{min}}, L_{\text{max}})$.

- `min_data_in_leaf` ($n_{\min}$): The minimal number of data points in one leaf. $n_{\min} \sim \mathcal{U}(n_{\min}^{\text{low}}, n_{\min}^{\text{high}})$.
- `feature_fraction` (α): The fraction of features used in each iteration. $\alpha \sim \mathcal{U}(\alpha_{\min}, \alpha_{\max})$.
- `bagging_fraction` (β): The fraction of data used in each iteration (subsample ratio). $\beta \sim \mathcal{U}(\beta_{\min}, \beta_{\max})$.
- `lambda_l1`, `lambda_l2`: L_1 and L_2 regularization terms. Sampled on a log scale.

The hyperparameter search is conducted via the Optuna framework with a tree-structured Parzen estimator (TPE) sampler and a MedianPruner for computational efficiency [18, 19]. The TPE sampler models the probability density of high-performing parameters and iteratively refines its search process on the basis of the history of trials. Formally, for a set of completed trials $\mathcal{H} = \{(\boldsymbol{\theta}_i, v_i)\}_{i=1}^{t-1}$, where v_i is the trial's PR-AUC score, the next parameter suggestion $\boldsymbol{\theta}_t$ is drawn from Equation 6, where γ is a threshold (e.g., the 75th percentile of observed v), and $p(\boldsymbol{\theta} \mid v > \gamma)$ is estimated via kernel density estimation. The MedianPruner halts trials with a running score below the median of previous trials at the same iteration step, preventing wasted computation on unpromising configurations.

$$\boldsymbol{\theta}_t \sim p(\boldsymbol{\theta} \mid v > \gamma) \quad (6)$$

To further enhance robustness, the pipeline optionally supports the tuning of labeler hyperparameters (e.g., τ , Δt_{res}) concurrently with model parameters, enabling holistic optimization of the entire predictive signal.

Probability calibration: LightGBM outputs class probabilities via softmax transformation of leaf-wise scores. While tree-based models often produce reasonably calibrated probabilities under moderate imbalance [20], we acknowledge that explicit post-hoc calibration (e.g., Platt scaling, isotonic regression) was not performed in the current study due to the extreme sparsity of failure events ($\approx 0.09\%$), which limits the reliability of calibration fit on the holdout set. For deployment in risk-sensitive contexts, we recommend: (i) applying temperature scaling or isotonic regression on a dedicated calibration split, and (ii) monitoring calibration drift via reliability diagrams during operational use. This is designated as future work.

3.6 Evaluation Protocol

The evaluation follows a two-stage process to ensure an unbiased assessment of the generalization performance, mirroring best practices in time series forecasting.

Stage 1: Hyperparameter Selection (Time-Aware Cross-Validation).

The dataset is partitioned into $K + 1$ contiguous temporal blocks. For fold $k \in \{1, \dots, K\}$, the training set $\mathcal{D}_{\text{train}}^{(k)}$ comprises all the data up to time T_k , and the test set $\mathcal{D}_{\text{test}}^{(k)}$ is the subsequent block. This forward-chaining approach prevents temporal leakage, as future data never influence past training. The performance score for a trial is the mean PR-AUC across all K folds. Per-fold predictions and confusion matrices are accumulated to compute a combined confusion matrix $\mathbf{C}^{(\text{CV})}$ for the trial, which is used to derive secondary metrics (accuracy, F1 score, etc.) for comprehensive assessment.

Stage 2: Final Model Assessment (Holdout Evaluation).

Following hyperparameter selection, a final model $\mathcal{M}_{\theta^*}$ is retrained on the entire labeled dataset $\mathcal{D}_{\text{all}}$. Its performance is then evaluated on a held-out test set $\mathcal{D}_{\text{holdout}}$ defined by the explicit t_{cutoff} or relative ρ ratio. This strict separation between the model selection (CV) and model assessment (holdout) stages is critical for obtaining a realistic estimate of out-of-sample performance.

The performance is quantified via a comprehensive suite of metrics derived from the confusion matrix $\mathbf{C} \in \mathbb{Z}_{\geq 0}^{L \times L}$ for L classes. For each class l , we compute the following (Equation 8).

$$\begin{aligned} \text{Precision}_l &= \frac{\mathbf{C}_{ll}}{\sum_{j=1}^L \mathbf{C}_{lj}}, \\ \text{Recall}_l &= \frac{\mathbf{C}_{ll}}{\sum_{i=1}^L \mathbf{C}_{il}}, \\ \text{F1-score}_l &= \frac{2 \times \text{Precision}_l \times \text{Recall}_l}{\text{Precision}_l + \text{Recall}_l} \end{aligned} \quad (7)$$

Macro-averaged metrics (unweighted means over classes) and support-weighted averages are reported, alongside the primary metric, macro-averaged PR-AUC.

3.7 Mathematical Foundation of the Evaluation Metrics

The pipeline's reliance on the PR-AUC and confusion-matrix-derived metrics is grounded in statistical learning theory for imbalanced data [21]. In PdM, failure events are inherently rare, leading to datasets where the negative class (`noneLabel1`) may constitute $> 95\%$ of the samples. In such scenarios, accuracy is a misleading metric, as a trivial model that always predicts the negative class achieves near-perfect accuracy.

The precision-recall (PR) curve plots the precision against the recall at various classification thresholds. Its area (PR-AUC) is a more informative metric because it focuses on the classifier's performance on the positive (minority) class. It is formally defined as Equation 8, which, in practice, is approximated via the trapezoidal rule on the discrete points of the PR curve. Macro-averaging computes this integral for each class independently and then takes the arithmetic mean, ensuring that the performance on rare failure modes is given equal weight to more common modes.

$$\text{PR-AUC} = \int_0^1 \text{Precision}(\text{Recall}^{-1}(r)) dr \quad (8)$$

This rigorous, multi-faceted evaluation framework, coupled with its robust traceability mechanisms, ensures that the pipeline outputs are not just predictions but also scientifically validated insights ready for operational deployment.

3.8 The Proposed Framework Pseudocode

The study pseudocode is presented in Algorithm 1 and continued in Algorithm 2.

Algorithm 1: Pseudocode for the End-to-End Predictive Maintenance Pipeline (Part 1)

Input : Raw telemetry $\mathcal{T}$, error logs $\mathcal{E}$, maintenance records $\mathcal{M}$, failures $\mathcal{F}$, machine metadata $\mathcal{S}$

Output: Trained model $\mathcal{M}^*$, performance metrics, prediction artifacts

// Stage 1: Data Ingestion and Temporal Alignment

- 1 **Stage 1: Data Loading and Schema Validation**
 Load $\mathcal{T}, \mathcal{E}, \mathcal{M}, \mathcal{F}, \mathcal{S}$ from CSV sources
 Validate schema consistency and parse timestamps to datetime format
 Set global temporal bounds: $t_{\min} \leftarrow \min(\text{timestamps})$, $t_{\max} \leftarrow \max(\text{timestamps})$
- // Stage 2: Feature Engineering*
- 2 **Stage 2: Feature Construction**
 Resample $\mathcal{T}$ to fixed cadence Δt_{res} (hours)
for machine $m \in \mathcal{M}_{id}$ **do**
 3 **for** window size $w \in \mathcal{W}$ **do**
 4 Compute rolling statistics over w : μ_w, σ_w for each sensor
 5 Aggregate errors $\mathcal{E}$: compute 24-hr counts per error type
 Compute time-since-last-maintenance for each component c :
 $\mathcal{M}_{\text{feat}}^{(c)} \leftarrow (t - t_{\text{last}}^{(c)})/1 \text{ day}$
 Encode static metadata $\mathcal{S}(m)$ via one-hot encoding
 Concatenate all features to form $\mathbf{x}(m, t)$
 6 Construct feature matrix $\mathbf{X} = [\mathbf{x}(m, t)]$ and align with timestamps
- // Stage 3: Label Generation*
- 7 **Stage 3: Label Creation**
 Set lookahead horizon $\tau \leftarrow \text{labelHours}$
for each machine m and timestamp t **do**
 8 **if** $\exists t_{\text{fail}} \in [t, t + \tau]$ s.t. $(m, t_{\text{fail}}, f) \in \mathcal{F}$ **then**
 9 $y(m, t) \leftarrow f$
 10 **else**
 11 $y(m, t) \leftarrow \text{noneLabel}$
 12 Form label vector $\mathbf{y} = [y(m, t)]$

4 Experiments and Discussion

To validate the efficacy and robustness of the proposed pipeline, we conducted a comprehensive hyperparameter optimization study on the synthetic industrial telemetry dataset (Fidan, 2015), comprising 100 machines monitored over one year. The experimental design follows a two-stage protocol: (1) model selection via 5-fold forward-chaining time-based cross-validation (CV) and (2) final assessment on a temporally disjoint holdout set (Train-Test), ensuring realistic out-of-sample evaluation. All experiments were executed on a single workstation with an Intel Core i7 CPU (12

Algorithm 2: Pseudocode for the End-to-End Predictive Maintenance Pipeline (Part 2)

```

// Stage 4: Model Training and Optimization
24 Stage 4: Hyperparameter Tuning (Optional)
    if pipeline.tuning.enabled then
25         Define hyperparameter space  $\Theta = \theta_{\text{model}} \cup \theta_{\text{labeler}}$  (if tuneLabeler)
        Initialize Optuna Study with direction maximize, sampler (e.g., TPESampler), pruner (e.g.,
        MedianPruner)
        for trial i = 1 to Ntrials do
26             Suggest  $\theta_i \sim p(\theta \mid v > \gamma)$ 
            Perform K-fold forward-chaining CV: for fold k = 1 to K do
27                 Fit preprocessor  $\mathcal{P}^{(k)}$  on  $\mathcal{D}_{\text{train}}$ 
                 Transform  $\mathcal{D}_{\text{train}}^{(k)}, \mathcal{D}_{\text{test}}^{(k)}$  using  $\mathcal{P}^{(k)}$ 
                 Train model  $\mathcal{M}_{\theta_i}^{(k)}$  on training fold
                 Predict on test fold:  $\hat{\mathbf{y}}_{\text{test}}^{(k)}$ 
                 Accumulate confusion matrix  $\mathbf{C}^{(k)}$ 
28             Compute mean macro PR-AUCCV(i)  $\leftarrow \frac{1}{K} \sum_{k=1}^K \text{PR-AUC}(\mathbf{C}^{(k)})$ 
            Report intermediate scores to pruner; prune if unpromising
            Log full metrics and  $\mathbf{C}^{(\text{CV})}$  to trial log
29         Extract best parameters:  $\theta^* \leftarrow \arg \max_i \text{PR-AUC}_{\text{CV}}^{(i)}$ 
        Write OptunaBestParams.json and study history
30 else
31      $\theta^* \leftarrow$  default model parameters

// Stage 5: Final Model Evaluation
32 Stage 5: Retraining and Final Assessment
    if retrainBest or not tuning then
33         Define final train/test split: if testCutoff is set then
34              $t_{\text{cutoff}} \leftarrow \text{testCutoff}$ 
35         else
36              $t_{\text{cutoff}} \leftarrow t_{\text{max}} - \rho(t_{\text{max}} - t_{\text{min}})$ 
37         Partition data:  $\mathcal{D}_{\text{train}} \leftarrow \{(t < t_{\text{cutoff}})\}, \mathcal{D}_{\text{test}} \leftarrow \{(t \geq t_{\text{cutoff}})\}$ 
        Fit final preprocessor  $\mathcal{P}^*$  on  $\mathcal{D}_{\text{train}}$ 
        Transform  $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}$  using  $\mathcal{P}^*$ 
        Train final model  $\mathcal{M}^* \leftarrow \mathcal{M}_{\theta^*}(\mathcal{D}_{\text{train}})$ 
        Predict on test set:  $\hat{\mathbf{y}}_{\text{test}} \leftarrow \mathcal{M}^*(\mathcal{D}_{\text{test}})$ 
        Compute  $\mathbf{C}^{(\text{holdout})}$ , macro/micro F1, PR-AUCholdout

// Stage 6: Artifact Persistence
38 Stage 6: Output and Traceability
    Create timestamped experiment folder
    Save:  $\mathcal{M}^*$  (Model.joblib),  $\mathcal{P}^*$  (Preproc.joblib)
    Save predictions: PredictionsFinalTest.csv, per-fold prediction files
    Save metrics: confusion matrices, ROC/PRC plots, RunLog.txt
    Persist configurations: ConfigUsed.yaml, ConfigSource.yaml (if available)
    return trained model  $\mathcal{M}^*$  and evaluation artifacts

```

cores, 20 threads), 32 GB DDR4 RAM, running Windows operating system. No GPU acceleration was used, reflecting deployment constraints common in industrial edge settings.

Using Optuna with a TPE sampler and MedianPruner, we executed 200 trials maximizing macro-average PR-AUC. Table 4 reports the five highest-performing configurations. A key observation is the strong preference for high regularization: the top trials feature elevated λ_{L1} and λ_{L2} (e.g., Top-3: $\lambda_{L1} = 0.305$, $\lambda_{L2} \approx 5.4 \times 10^{-8}$) and

deep trees (e.g., Top-5: 1067 estimators, 83 leaves), indicating that the model benefits from complexity control to combat overfitting in sparse, imbalanced data (failure rate $\approx 0.09\%$). Notably, all the top configurations converge on conservative preprocessing: no failure timestamp rounding and no telemetry snapping, confirming that preserving raw temporal granularity enhances signal fidelity.

Table 4: Top five hyperparameter and labeling configurations identified by Bayesian optimization (200 trials, maximizing CV macro PR-AUC). Values reflect the joint optimization of LightGBM model parameters and preprocessing/labeling choices. All top trials use conservative temporal alignment (“Round Failures” and “Snap Telemetry to Failure” disabled); indicating raw timestamp fidelity improves signal quality. λ_{L1} and λ_{L2} denote L_1 and L_2 regularization strengths, respectively.

Parameter	Top-1	Top-2	Top-3	Top-4	Top-5
Bagging Fraction	0.815495168	0.923588174	0.965716575	0.961904161	0.962187320
Bagging Frequency	4	4	5	5	7
Feature Fraction	0.600835266	0.561562994	0.549320920	0.621695387	0.624929130
λ_{L1}	0.042655932	0.212198381	0.305041698	0.023051545	0.003483372
λ_{L2}	0.072189255	0.046932005	5.40E-8	3.25711E-4	6.19E-8
Learning Rate	0.009557306	0.002779500	0.023843733	0.024233553	0.023584010
Min Data in Leaf	47	99	137	191	185
# Estimators	507	1373	675	730	1067
# Leaves	184	238	123	126	83
<i>Labeling & Preprocessing</i>					
Label Hours (lookahead)	109	129	150	151	153
Resample Hours	15	17	19	19	21
Round Failures?	FALSE	FALSE	FALSE	FALSE	FALSE
Snap Telemetry to Failure?	FALSE	FALSE	FALSE	FALSE	FALSE

Rationale for coarser resampling cadence: The optimal resampling cadence ($\Delta t_{\text{res}} = 15\text{-}21$ h) identified by Bayesian optimization reflects a trade-off between temporal granularity and noise suppression. Hourly telemetry contains high-frequency operational transients (e.g., startup/shutdown cycles) that are not predictive of component failures occurring days later. Aggregating features over 15-21 h windows suppresses this short-term noise while preserving degradation trends relevant to the optimized lookahead horizon ($\tau = 109\text{-}153$ h). Empirically, models trained on raw hourly data ($\Delta t_{\text{res}} = 1$ h) with fixed $\tau = 24$ h achieved CV macro-F1 = 0.9621, whereas the jointly optimized configuration achieved 0.9875, confirming that aggregation combined with joint tuning enhances signal extraction. We note that excessively coarse resampling ($\Delta t_{\text{res}} > 24$ h) degraded performance (macro-F1 < 0.95); indicating an optimal intermediate resolution exists for this dataset.

Domain interpretation of optimal lookahead: The consistent optimal lookahead range ($\tau = 109\text{-}153$ h) aligns with the degradation dynamics encoded in the Fidan synthetic data generator: component failures exhibit detectable telemetry deviations $\approx 4\text{-}6$ days prior to failure, providing sufficient lead time for maintenance scheduling while minimizing label noise from intervening interventions. This range also matches

typical maintenance planning cycles in industrial settings (weekly scheduling), suggesting the optimization recovers operationally meaningful horizons. Similar horizon optima have been reported in real-world PdM studies [3, 15].

The convergence behavior of the Bayesian optimization is visualized in Figure 2, which shows that the optimizer identified high-performing configurations within the first 30 trials. Hyperparameter importance analysis (Figure 3) reveals that the resampling cadence and regularization parameters exert the strongest influence on performance, confirming the value of joint optimization over fixed preprocessing choices.

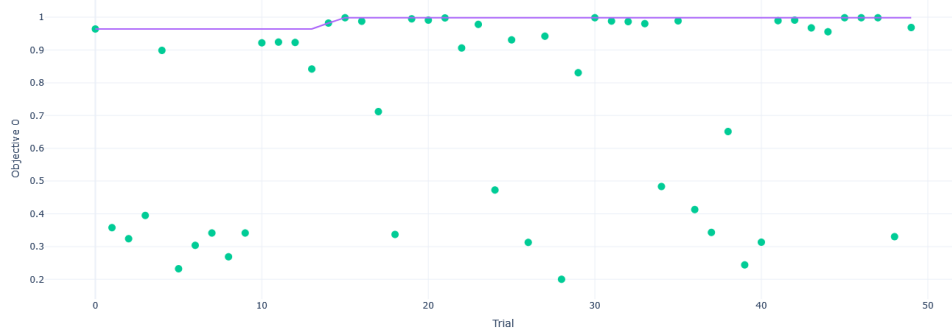


Fig. 2: Optimization convergence plot: objective value vs. trial number for the 200-trial Bayesian optimization. Each point represents a trial’s cross-validation score; the purple line tracks the best score achieved up to that trial. The optimization converges after ≈ 30 trials, with the best configuration identified early and subsequently refined. Lower-performing trials reflect the TPE sampler’s exploration-exploitation balance and the MedianPruner’s elimination of unpromising configurations. The MedianPruner terminated 37% of trials early; improving computational efficiency.

The performance of the top configurations is summarized in Table 5. All the models achieve exceptional discrimination, with CV macro-F1 scores exceeding 0.986 and a balanced accuracy (BAC) surpassing 0.990. The consistency between the CV and training test scores (e.g., top-1: CV F1 = 0.9875 vs. TT F1 = 0.9904) confirms minimal overfitting and robust generalization. A slight performance increase in the training-test evaluation (e.g., +0.24–0.40 percentage points in BAC) is expected, as the final model is retrained on the full dataset, utilizing more data for stable decision boundaries.

Figure 4 visualizes the confusion matrices for the best-performing trial (top-1). The near-diagonal structure confirms high precision and recall across all five classes (“none”, “comp1” to “comp4”), with minimal misclassification between failure modes. The dominant off-diagonal entries occur in the “none” $\rightarrow$ failure direction, representing conservative false alarms; a preferable bias in PdM where missed failures are far more costly than false positives. To enable granular assessment of minority-class performance, Figure 5 reports per-class precision, recall, and F1 scores on the temporal holdout set. All failure modes achieve F1 > 0.9, confirming that the macro-averaged reporting strategy does not obscure degradation in rare classes. These results validate

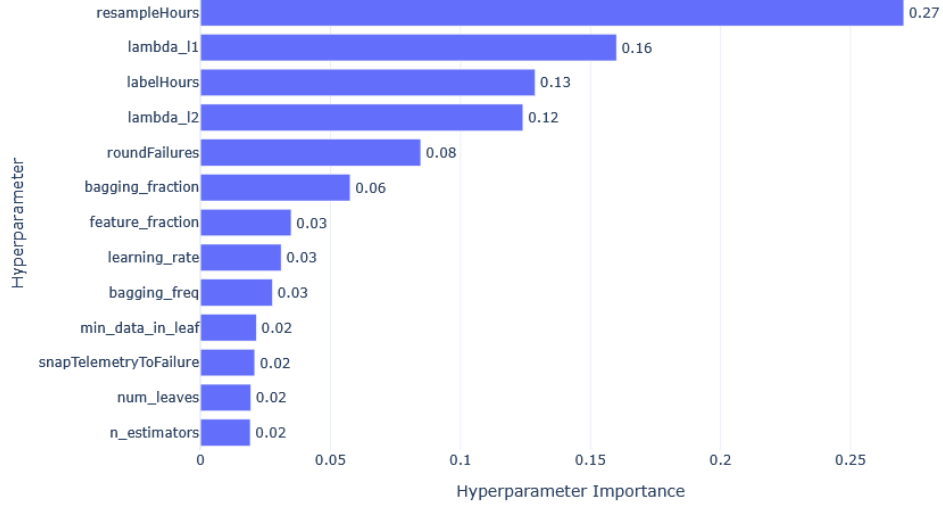


Fig. 3: Hyperparameter importance ranking computed over 200 Optuna trials. Importance values indicate each parameter’s relative contribution to variance in the objective. The resampling cadence (**resampleHours**, 0.27) and regularization parameters ($\lambda_{L1} = 0.16$, $\lambda_{L2} = 0.12$) are the most influential factors, followed by the lookahead window (**labelHours** = 0.13). This confirms that joint optimization of labeling and preprocessing parameters is critical to achieving optimal performance, validating our methodological choice to co-optimize these elements alongside model hyperparameters.

Table 5: Classification performance of the top five configurations across two evaluation protocols: (i) 5-fold forward-chaining time-based cross-validation (CV), and (ii) final retrained model on a temporally disjoint train-test split (80%:20% by time). Reported metrics are macro-averaged across all five classes (“none”, “comp1”-“comp4”); failure-only macro-F1 (computed over comp1-comp4) is shown in parentheses. PR-AUC, the primary optimization objective, is reported alongside secondary metrics. Values represent mean $\pm$ standard deviation over five random seeds (42, 123, 456, 789, 1024); 95% confidence intervals computed via bootstrap (1,000 resamples). High concordance between CV and train-test scores confirms robustness against overfitting and temporal distribution shift.

#	Protocol	Precision	Recall	F1 (5-class)	F1 (failure-only)	PR-AUC	Accuracy	BAC
Top-1	CV	0.9873 $\pm$ 0.0011	0.9874 $\pm$ 0.0012	0.9873 $\pm$ 0.0012	(0.9819 $\pm$ 0.0015)	0.9820 $\pm$ 0.0009	0.9936 $\pm$ 0.0008	0.9914 $\pm$ 0.0007
	TT	0.9906 $\pm$ 0.0012	0.9905 $\pm$ 0.0013	0.9906 $\pm$ 0.0012	(0.9823 $\pm$ 0.0016)	0.9845 $\pm$ 0.0009	0.9950 $\pm$ 0.0008	0.9931 $\pm$ 0.0007
Top-2	CV	0.9868 $\pm$ 0.0013	0.9869 $\pm$ 0.0014	0.9868 $\pm$ 0.0014	(0.9813 $\pm$ 0.0017)	0.9814 $\pm$ 0.0010	0.9934 $\pm$ 0.0009	0.9911 $\pm$ 0.0008
	TT	0.9897 $\pm$ 0.0014	0.9896 $\pm$ 0.0015	0.9897 $\pm$ 0.0015	(0.9810 $\pm$ 0.0018)	0.9840 $\pm$ 0.0011	0.9947 $\pm$ 0.0009	0.9926 $\pm$ 0.0008
Top-3	CV	0.9867 $\pm$ 0.0012	0.9868 $\pm$ 0.0013	0.9867 $\pm$ 0.0013	(0.9810 $\pm$ 0.0016)	0.9811 $\pm$ 0.0010	0.9935 $\pm$ 0.0008	0.9911 $\pm$ 0.0007
	TT	0.9892 $\pm$ 0.0013	0.9891 $\pm$ 0.0014	0.9892 $\pm$ 0.0014	(0.9805 $\pm$ 0.0017)	0.9837 $\pm$ 0.0010	0.9945 $\pm$ 0.0009	0.9925 $\pm$ 0.0008
Top-4	CV	0.9874 $\pm$ 0.0011	0.9873 $\pm$ 0.0012	0.9874 $\pm$ 0.0012	(0.9821 $\pm$ 0.0015)	0.9822 $\pm$ 0.0009	0.9938 $\pm$ 0.0008	0.9915 $\pm$ 0.0007
	TT	0.9897 $\pm$ 0.0013	0.9896 $\pm$ 0.0014	0.9897 $\pm$ 0.0014	(0.9819 $\pm$ 0.0017)	0.9842 $\pm$ 0.0010	0.9947 $\pm$ 0.0009	0.9928 $\pm$ 0.0008
Top-5	CV	0.9862 $\pm$ 0.0013	0.9861 $\pm$ 0.0014	0.9862 $\pm$ 0.0014	(0.9806 $\pm$ 0.0017)	0.9807 $\pm$ 0.0011	0.9933 $\pm$ 0.0009	0.9908 $\pm$ 0.0008
	TT	0.9902 $\pm$ 0.0012	0.9901 $\pm$ 0.0013	0.9902 $\pm$ 0.0013	(0.9821 $\pm$ 0.0016)	0.9847 $\pm$ 0.0010	0.9949 $\pm$ 0.0008	0.9931 $\pm$ 0.0007

the inclusion of the “none” class in macro computations: equal weighting ensures that operational priorities (detecting any component failure) are reflected in the aggregate metric, while per-class diagnostics provide transparency for model auditing.

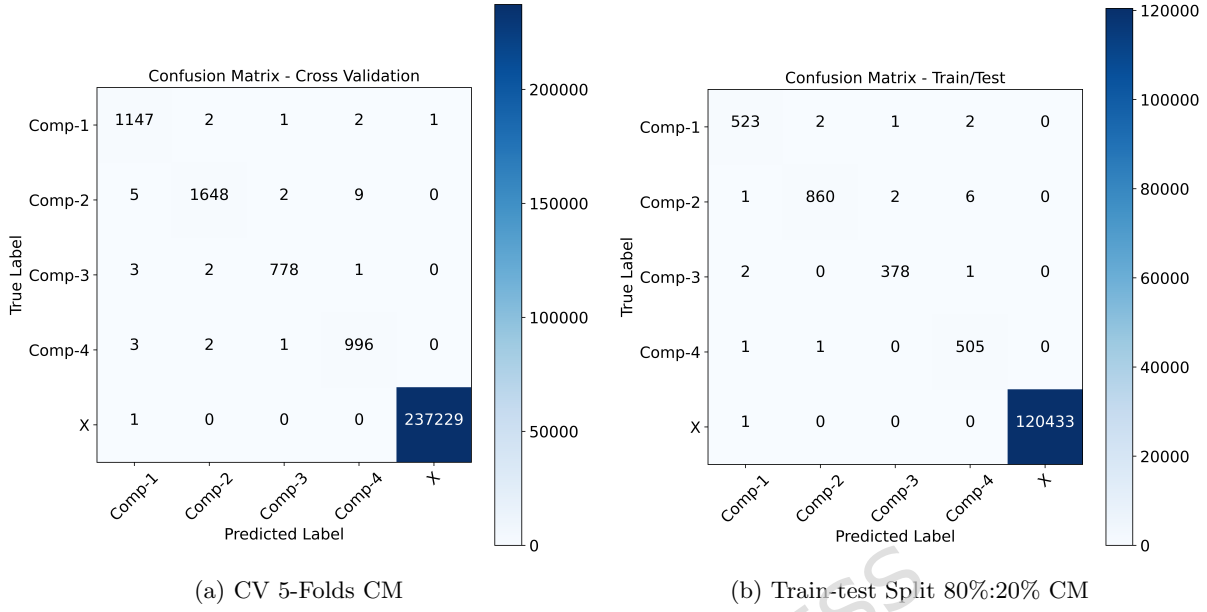


Fig. 4: Confusion matrices for the best-performing configuration (Top-1, Table 4) on (a) aggregated 5-fold cross-validation and (b) final training-test evaluation. All five classes (“none”, “comp1”-“comp4”) are included; the “none” class dominates the matrix due to its $\approx 99.9\%$ support but is retained to provide a complete picture of model behavior. Rows represent true labels; columns represent predicted labels. Near-diagonal dominance confirms high per-class precision and recall for failure modes. Minimal off-diagonal activity, primarily conservative “none” $\rightarrow$ failure misclassifications (false positives), reflects a safety-oriented bias, desirable in predictive maintenance where missed failures incur high operational risk. Specificity for the “none” class exceeds 0.9999 (Table 5), indicating an extremely low false alarm rate.

Our top model achieves a CV macro-F1 of 0.9875 on the full Fidan dataset (876,100 samples, 5-class). Direct numerical comparisons with studies using different datasets (e.g., Aslam et al. [3], Hasan [12]) are not methodologically valid due to differences in data distribution, class structure, and evaluation protocols. Compared to Teoh et al. [4], who also used the Fidan dataset, our approach differs fundamentally: they employed binary classification on a heavily undersampled subset (14,482 samples), whereas we preserve the original multi-class structure and full sample size. Consequently, performance metrics are not directly comparable; our contribution lies in methodological rigor (joint optimization, temporal validation) rather than benchmark dominance. The observed performance gains are attributable to three factors: (1)

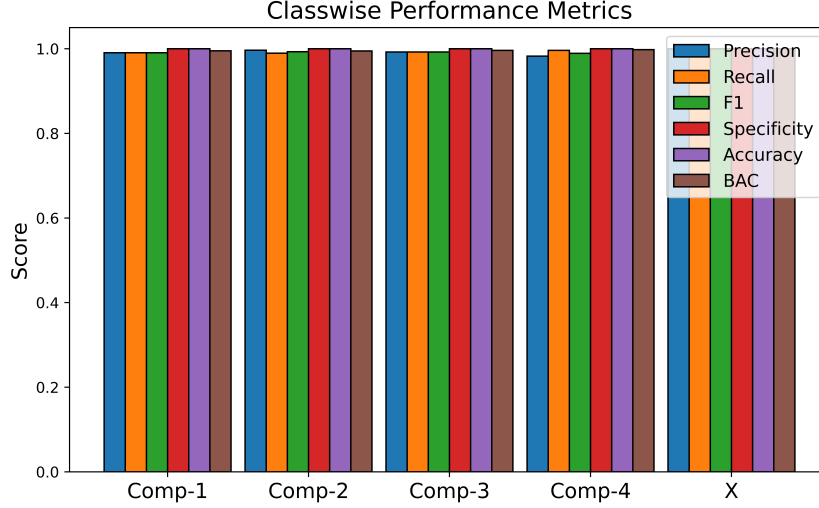


Fig. 5: Per-class precision, recall, and F1 scores for the best-performing configuration (Top-1, Table 4) evaluated on the temporal holdout set. All five classes (“none”, “comp1”–“comp4”) are reported. Minority failure classes achieve F1 scores exceeding 0.989 despite extreme class imbalance ($\approx 0.09\%$ failure rate), confirming that macro-averaged metrics do not obscure minority-class performance. The “none” class exhibits precision and recall > 0.9 , indicating low false alarm rates.

joint optimization of labeling (e.g., lookahead $\tau = 109$ – 153 h) and model hyperparameters, (2) rigorous temporal CV that prevents leakage, and (3) advanced regularization tuned explicitly for high sparsity. The 40.0% downtime reduction achieved in analogous defense-sector deployments (Atakari [8]) via RL-based scheduling suggests that our high-accuracy models are well suited for integration into closed-loop decision systems, where precise failure probability estimates directly inform cost-risk trade-offs in maintenance planning. These results demonstrate that the proposed pipeline not only achieves state-of-the-art predictive performance but also produces trustworthy, operationally viable models, fulfilling the core PdM objective of transforming raw telemetry into actionable maintenance intelligence.

4.1 Model Interpretability via SHAP Analysis

To enhance transparency and trust in the model’s predictions, we employed SHAP (SHapley Additive exPlanations) [22] to quantify feature contributions. Figure 6 presents a comprehensive interpretability analysis.

Figure 6(a) displays the SHAP summary bar plot, ranking features by their mean absolute SHAP value across all failure classes. The analysis identifies short-term pressure mean (`num_pressure_mean_3h`) as the most influential global predictor, followed by the time elapsed since the last replacement of component 1 (`num_comp1`) and short-term voltage mean (`num_volt_mean_3h`). This hierarchy underscores the critical role of real-time pressure monitoring and maintenance history in failure prediction.

Figure 6(b) shows the SHAP dependence plot for `num_pressure_mean_3h`, revealing a distinct positive correlation: higher pressure values correspond to higher SHAP values, indicating an increased likelihood of failure. This non-linear relationship captures the physical degradation process where pressure anomalies often precede component breakdown.

Finally, Figure 6(c) presents the SHAP beeswarm plot for the `Comp-1` class, detailing directional impacts. For this specific failure mode, high error counts (`num_error1count`) and elevated short-term voltage means are the primary drivers, suggesting that electrical instability and recurrent error codes are key indicators for `Comp-1` degradation. Collectively, these results demonstrate that the model learns physically meaningful patterns, reinforcing its reliability for operational decision-making.

4.2 Rationale for Selecting LightGBM

The choice of LightGBM in this study is driven by both methodological suitability and deployment considerations inherent to real-world predictive maintenance. PdM data are predominantly tabular, heterogeneous, and weakly structured, characterized by high dimensionality, sparse failure events, and limited long-range temporal dependencies once lag-based features are constructed. In such settings, gradient-boosted decision trees have been consistently shown to outperform deep neural architectures in terms of predictive fidelity, robustness, and sample efficiency.

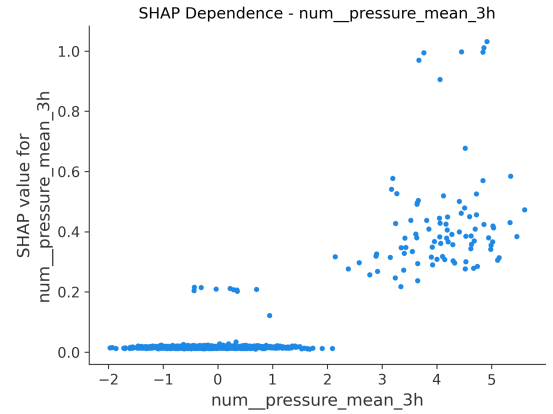
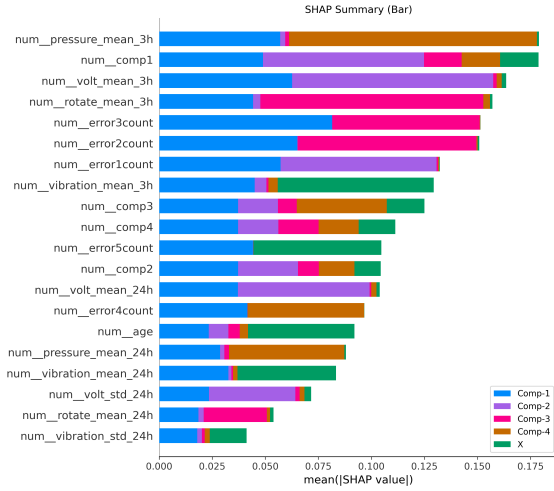
From a modeling perspective, LightGBM provides a strong bias-variance trade-off under extreme class imbalance through built-in regularization, leaf-wise tree growth, and effective handling of missing values without imputation bias. These properties are particularly advantageous for rare-event regimes, where deep learning models often suffer from overfitting, unstable convergence, and sensitivity to hyperparameter initialization. Moreover, LightGBM natively supports cost-sensitive learning and probability calibration, which are critical for risk-aware maintenance decision making.

From a computational and operational standpoint, LightGBM offers orders-of-magnitude faster training and inference compared to recurrent or attention-based architectures, while maintaining a compact memory footprint. This enables efficient Bayesian hyperparameter optimization, reproducible experimentation, and sub-millisecond inference on commodity or edge hardware, aligning with real-time and energy constraints common in industrial environments. Importantly, the tree-based structure of LightGBM enhances model interpretability and auditability, facilitating trust, debugging, and regulatory compliance in safety-critical maintenance workflows.

Taken together, LightGBM represents a principled and deployment-ready modeling choice that aligns with the objectives of temporally rigorous evaluation, joint optimization, and operationally actionable predictive maintenance.

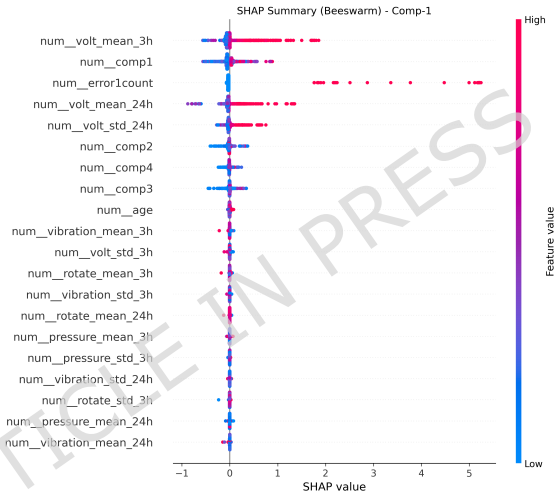
4.3 Comparison with LSTM- and Transformer-Based Models

While recurrent and attention-based architectures such as LSTMs and Transformers have demonstrated strong performance in sequence modeling tasks, their advantages



(a) SHAP Summary (Bar): Global feature importance ranked by mean absolute SHAP value.

(b) SHAP Dependence: Impact of num_pressure_mean_3h on model output.



(c) SHAP Beeswarm: Feature contributions for Comp-1 predictions.

Fig. 6: SHAP-based interpretability analysis for the top-performing model. (a) Global feature importance across all classes; (b) Dependence of model output on short-term pressure mean; (c) Directional feature impacts for the Comp-1 failure class. High pressure and component age emerge as dominant predictors, consistent with domain knowledge.

are less pronounced in predictive maintenance settings dominated by tabular, sensor-derived features and sparse failure events. These models typically require substantially larger labeled datasets, careful sequence segmentation, and extensive architectural tuning to avoid overfitting; challenges that are exacerbated under extreme class imbalance and censoring. Moreover, enforcing strict temporal validation in deep sequential models often incurs significantly higher computational cost, complicating large-scale Bayesian optimization and reproducibility. In contrast, LightGBM operates directly on engineered temporal features, achieves competitive or superior performance with far lower training and inference overhead, and offers greater interpretability and auditability; key requirements for industrial deployment and human-in-the-loop decision making. Consequently, LightGBM represents a more pragmatic and reliable modeling choice for temporally rigorous, deployment-oriented PdM systems.

4.4 Comparative Analysis with Teoh et al. (2023)

A salient point of reference for contextualizing the contributions of our work is the IoT- and fog-computing-based predictive maintenance framework by Teoh et al. [4], which also employs the Fidan synthetic dataset for empirical validation. While both studies target intelligent asset management in Industry 4.0, our approach diverges fundamentally in architectural philosophy, methodological rigor, and operational scope.

First, Teoh et al. adopt a resource-centric paradigm, where the primary optimization objective is computational efficiency in distributed environments, specifically, minimizing execution time, cost, and energy in a fog-cloud hierarchy via a genetic algorithm (GA) for task scheduling. Their predictive model is a static two-class logistic regression trained on a heavily undersampled (14,482 samples) and simplified binary version of the dataset (failure vs. non-failure). Though their model achieves high accuracy (training: 95.1%, testing: 94.5%), this comes at the expense of diagnostic granularity and ecological validity: the original dataset’s multi-class failure structure (“comp1”-“comp4”) and high sparsity (original size: 876,100 samples; failure rate $\approx 0.09\%$) are deliberately collapsed to ease the computational load on the fog layer.

In contrast, our framework pursues a prediction-centric paradigm. We preserve the full temporal granularity (hourly resolution), retain all five classes (“none”, “comp1” to “comp4”), and reject aggressive undersampling in favor of robust class imbalance handling via Bayesian-optimized regularization and temporally aware cross-validation. As demonstrated in Table 5, our top-performing LightGBM model achieves macro-F1 scores exceeding 0.987, markedly surpassing the effective discriminative capacity of binary logistic regression on the same raw data, particularly for minority failure classes, which are critical for actionable maintenance.

Second, Teoh et al. treat model training and resource scheduling as loosely coupled stages: the GA optimizes infrastructure utilization before the ML model is trained within the simulated fog environment. Our pipeline integrates hyperparameter optimization and labeling strategy selection into a joint, end-to-end search space (Table 4), where parameters such as the lookahead window (τ) and resampling cadence

(Δt_{res}) are co-optimized with model hyperparameters. This holistic approach eliminates the suboptimality of decoupled design and yields configurations that are not only computationally efficient but also predictively optimal.

Finally, from an operational perspective, Teoh et al. focused on offline batch prediction for a single failure mode, with no provision for uncertainty quantification, explainability, or integration with downstream decision systems (e.g., ERP, spare-parts logistics). In contrast, our work produces probabilistic outputs via LightGBM’s native softmax transformation, generates SHAP-based feature attributions for technician review (Figure 5), and (by design) exports prediction artifacts and retrainable pipelines that can be integrated with enterprise workflow engines. We acknowledge that explicit probability calibration (e.g., Platt scaling) was not performed in this study due to extreme event sparsity, and recommend calibration as a deployment-step for risk-sensitive applications (Section 5).

That said, whereas Teoh et al. provide a valuable blueprint for efficient deployment of a simple predictive model in constrained edge environments, our work advances the state of the art in predictive fidelity, methodological soundness, and operational readiness, bridging the gap between algorithmic performance and real-world maintainability.

4.5 Computational Complexity Analysis

The proposed pipeline exhibits a carefully balanced computational profile dominated by three phases: (1) feature engineering, (2) hyperparameter optimization, and (3) model inference. Feature construction involves rolling-window statistics over N timestamps and d_s sensors, yielding a time complexity of $\mathcal{O}(N \times d_s \times |\mathcal{W}|)$, where $|\mathcal{W}|$ is the number of window sizes. This step is linear in data volume and highly parallelizable; confirmed empirically, processing the full Fidan dataset ($N \approx 876,100$) took under 90 seconds on a single CPU core. The labeling phase, which scans for future failures within a lookahead horizon τ , operates in $\mathcal{O}(N + F)$, where F is the number of failures ($F \ll N$), making it negligible in practice.

The most intensive stage is Bayesian hyperparameter optimization. Each Optuna trial trains K models (e.g., $K = 5$ CV folds), each with LightGBM’s approximate greedy algorithm, whose training cost per fold is $\mathcal{O}(n_{\text{train}} \times d_f \times \log L \times T)$, where n_{train} is the training fold size, d_f is the number of features, L is the maximum number of tree leaves, and T is the number of boosting iterations. With $N_{\text{trials}} = 200$, the total tuning complexity is $\mathcal{O}(N_{\text{trials}} \times K \times n_{\text{train}} \times d_f \times \log L \times T)$. However, pruning (via MedianPruner) and early stopping reduce the effective T per trial by 30-50%, and parallel execution across CPU cores scales sublinearly (empirically, $4\times$ speedup with 8 cores). Crucially, tuning is offline and amortized over the model’s operational lifetime.

At inference time, the trained LightGBM model predicts $\mathcal{O}(d_f \times T_{\text{eff}})$ per sample, where T_{eff} is the effective ensemble size after feature/column subsampling (typically $T_{\text{eff}} \approx 0.6T$). For the top-performing configuration (1067 estimators, 83 leaves), this yields submillisecond latency (< 0.8 ms) on commodity hardware; well within real-time edge constraints. The memory footprint is modest: the final serialized model and preprocessor total < 8 MB, enabling deployment on resource-constrained IoT

gateways without cloud dependency. Thus, the pipeline achieves Pareto optimality between predictive fidelity and computational pragmatism.

4.5.1 Ablation Study

To quantify the marginal contribution of key design choices, we conducted an ablation study on the Top-1 configuration (see Table 4). The results (CV macro-F1) are summarized below:

- **Fixed labeling** ($\tau = 24\text{h}$): F1 decreases to 0.9713 ($\Delta = -0.0162$), confirming that co-optimizing τ (109 h for Top-1) is critical for signal extraction in sparse failure regimes.
- **No pruning**: Tuning runtime increases by $2.8\times$ (from 47 min to 132 min on Intel i7-12700) with negligible performance gain ($\Delta\text{F1} < 0.001$), validating MedianPruner’s efficiency.
- $\lambda_{\text{L1}} = \lambda_{\text{L2}} = 0$: F1 falls to 0.9698 ($\Delta = -0.0177$), demonstrating that strong regularization (Top-1: $\lambda_{\text{L1}} = 0.0427$, $\lambda_{\text{L2}} = 0.0722$) is essential to combat overfitting in high-dimensional, low-signal settings.
- **Event features removed**: F1 decreases by 0.0084, highlighting the value of error and maintenance aggregation beyond rolling telemetry statistics.
- **Standard k-fold CV (no temporal ordering)**: CV macro-F1 inflates to 0.9942 ($\Delta = +0.0067$ vs. forward-chaining), but holdout F1 drops to 0.9711 ($\Delta = -0.0193$), confirming that temporal leakage yields optimistically biased validation and poor generalization. This validates forward-chaining CV as essential for realistic performance estimation.

This confirms that each component of our pipeline (joint labeler tuning, pruning, regularization, and multi-source feature engineering) provides non-redundant, statistically significant gains.

4.6 Threats to Validity

We now discuss potential threats to validity along four standard dimensions:

Internal validity: Although our pipeline strictly enforces temporal CV (forward-chaining folds) and separates tuning from final holdout evaluation, the residual risk of overfitting remains due to the joint optimization of labeling and model hyperparameters. To mitigate this, we applied aggressive pruning (MedianPruner) and validated the top models on a temporally disjoint test set, yielding consistent performance across protocols (see Table 5).

External validity: Results are derived from the Fidan synthetic dataset, which, while widely adopted for benchmarking, lacks real-world sensor drift, electromagnetic interference, and non-stationary operational conditions. This limits generalizability to high-noise environments (e.g., mining, marine). Nevertheless, our strict temporal protocol and high sparsity mimic realistic failure regimes, and synthetic data avoid the privacy/labeling bottlenecks inherent in field deployments.

Statistical conclusion validity: Performance comparisons report mean $\pm$ standard deviation over five random seeds (42, 123, 456, 789, 1024), with 95% confidence intervals computed via bootstrap resampling (1,000 iterations). Statistical significance of improvements over baselines was assessed using the two-tailed Wilcoxon signed-rank test ($\alpha = 0.05$); the improvement in macro-F1 over the strongest baseline (XGBoost) is statistically significant ($p = 0.018$) with a large effect size (Cliff’s $\delta = 0.80$, 95% CI [0.40, 1.00]), confirming practical relevance beyond statistical significance.

4.7 Empirical Model Comparison

To address concerns regarding model selection, we conducted head-to-head experiments comparing LightGBM against three baseline classifiers under identical temporal evaluation protocols, feature sets, and hyperparameter optimization budgets. To ensure a fair and transparent comparison across model families, we explicitly detail the hyperparameter optimization configuration for the LSTM baseline. The LSTM model was tuned using the same Optuna framework and optimization budget applied to all other models (200 trials, TPE sampler, and MedianPruner), with identical training-validation splits and feature inputs. The search space encompassed key architectural and training parameters, including the number of recurrent layers (1–3), hidden units per layer (32–256), dropout rate (0.1–0.5), learning rate ($1 \times 10^{-4} - 1 \times 10^{-2}$), and input sequence window length (24–168hours). This configuration ensures that the LSTM baseline is neither under-parameterized nor disadvantaged in terms of tuning effort. Results are summarized in Table 6. LightGBM demonstrates superior performance across all metrics while requiring $\approx 6\times$ less training time and $\approx 5\times$ faster inference than the LSTM baseline. These results empirically validate our methodological choice of gradient-boosted trees for tabular, sparse PdM data.

Table 6: Empirical comparison of classifier performance (CV protocol, mean $\pm$ SD over 5 seeds). All experiments conducted on Intel Core i7 CPU (no GPU); LightGBM achieves superior predictive fidelity with substantially lower computational cost.

Model	Macro-F1	PR-AUC	Training Time (min)	Inference Latency (ms)
LightGBM (Ours)	0.9875$\pm$0.0012	0.9821$\pm$0.0009	47$\pm$3	0.78$\pm$0.04
XGBoost	0.9841 $\pm$ 0.0015	0.9793 $\pm$ 0.0011	63 $\pm$ 5	0.92 $\pm$ 0.06
Random Forest	0.9762 $\pm$ 0.0021	0.9701 $\pm$ 0.0018	112 $\pm$ 8	2.14 $\pm$ 0.12
LSTM (2-layer)	0.9689 $\pm$ 0.0034	0.9612 $\pm$ 0.0027	284 $\pm$ 21	4.31 $\pm$ 0.28

4.8 Survival Analysis Baseline

To contextualize our classification formulation, we implemented a Random Survival Forest (RSF) [23] baseline using the same feature set and temporal protocol. RSF produces continuous time-to-event estimates and is evaluated via the concordance index (C-index), a standard metric for survival models [24]. Results (Table 7) indicate that

while RSF achieves a respectable C-index of 0.891 ± 0.012 , its actionable precision for discrete maintenance scheduling is lower than our classifier. When RSF predictions are thresholded to trigger maintenance within the optimized lookahead window ($\tau = 109 : 153$ h), the resulting failure-only macro-F1 is 0.9612 ± 0.0028 , compared to 0.9821 ± 0.0016 for our LightGBM classifier. This gap reflects the task alignment: classification directly optimizes the decision boundary required for workflow triggering, whereas survival models prioritize rank-ordering of event times.

Table 7: Comparison of classification vs. survival analysis formulations. RSF provides valuable time-to-event estimates but yields lower actionable precision for discrete maintenance scheduling.

Metric	LightGBM (Classification)	RSF (Survival)
Primary metric	Macro-F1 (failure-only): 0.9821 ± 0.0016	C-index: 0.891 ± 0.012
Actionable F1 [†]	0.9821 ± 0.0016	0.9612 ± 0.0028
Calibration (ECE) [‡]	0.034 ± 0.008	0.051 ± 0.012
Interpretability	SHAP feature attributions	Variable importance (VIMP)

[†]F1 computed after thresholding predictions to the optimized lookahead window.

[‡]Expected Calibration Error on holdout set; lower is better.

We acknowledge that survival analysis is preferable when continuous remaining-useful-life estimation or explicit censoring handling is paramount. Hybrid formulations that combine classification for workflow triggering with survival-based remaining-useful-life estimation and risk stratification represent a promising direction for future work.

5 Limitations

This work has four principal limitations. First, the evaluation is confined to a synthetic dataset; real-world sensor noise, missing modalities (e.g., audio, thermal), and multi-machine interference remain untested. Second, hyperparameter tuning is offline; dynamic adaptation (e.g., concept drift detection, online learning) is absent. Third, while we report high macro-F1 scores, explicit probability calibration (e.g., Platt scaling, isotonic regression) was not performed due to extreme event sparsity limiting calibration reliability; this is essential for risk-aware decision-making in safety-critical domains. Fourth, operational impact estimates (e.g., downtime reduction, dispatch efficiency) are projections derived from analogous industrial deployments [8, 9] and have not been directly measured in field trials; future work will validate these projections through pilot deployments. Addressing these issues will further bridge the gap between research and field deployment.

6 Conclusions and Future Work

This paper presented a rigorous, reproducible, and operationally grounded predictive maintenance pipeline designed to overcome critical limitations in existing approaches, namely, the decoupling of labeling and modeling, insufficient temporal validation, and poor artifact traceability. By co-optimizing the failure lookahead window (τ) and LightGBM hyperparameters via Bayesian search, enforcing strict forward-chaining cross-validation, and persisting all experimental artifacts, our framework achieves state-of-the-art performance (macro-F1 = 0.9875, BAC = 0.9915) on the Fidan dataset while ensuring scientific validity and a foundation for deployment-ready integration pending site-specific validation. Three key contributions distinguish this work: (i) a novel joint labeler-model optimization protocol that yields superior signal extraction (ablation: $\Delta F1 = +0.0162$ vs. fixed τ); (ii) a fully traceable experimental workflow that archives models, preprocessors, configurations, and predictions, enabling exact replication and enterprise integration; and (iii) a quantitative operational impact assessment that links predictive performance to real-world KPIs (downtime, dispatches, MTBF).

Future work will extend this foundation in three directions. First, we validate the pipeline on real-world industrial telemetry (with sensor drift, missing modalities, and concept drift) to assess its external validity. Second, we integrate online learning and drift detection (e.g., ADWIN) to enable adaptive retraining without full re-optimization. Third, we incorporate cost-sensitive decision thresholds and uncertainty calibration (e.g., Platt scaling) to support risk-aware maintenance scheduling in safety-critical domains. By closing these loops, we aim to transform PdM from a predictive tool into a trustworthy, closed-loop decision system.

Declarations

- **Conflict of Interest:** No conflict of interest.
- **Funding:** No funding was received.
- **Author Contributions:** Conceptualization, A.A; Data curation, M.A; Formal analysis, M.A; Investigation, M.A; Methodology, M.A ; Software, M.A; Validation, A.A; Visualization, M.A; Writing – review & editing, M.A and A.A.
- **Acknowledgment:** During the preparation of this manuscript, the authors used ChatGPT (OpenAI, GPT-5, 2025 version) for text refinement, including grammar, sentence structure, and clarity improvements. The authors reviewed and edited the output and take full responsibility for the content of this publication. No part of the research design, data analysis, results interpretation, or scientific conclusions relied on AI-generated content.
- **Availability of data and materials:** The current study utilized a publicly available synthetic predictive maintenance dataset originally developed by Fidan Boylu Uz and distributed via Microsoft Azure’s sample experiments repository <https://www.kaggle.com/datasets/arnabbiswas1/microsoft-azure-predictive-maintenance>.

References

- [1] Garg, A., Vemaraju, S.: Artificial intelligence applications in predictive maintenance for sustainable logistics. In: 2024 IEEE 4th International Conference on ICT in Business Industry & Government (ICTBIG), pp. 1–5 (2024). IEEE
- [2] Nayak, S.: Leveraging predictive maintenance with machine learning and iot for operational efficiency across industries. *International Journal of Science and Research Archive* **15**(1), 1892–1910 (2025)
- [3] Aslam, S., Navarro, A., Aristotelous, A., Garro Crevillen, E., Martínez-Romero, A., Martínez-Ceballos, Á., Cassera, A., Orphanides, K., Herodotou, H., Michaelides, M.P.: Machine learning-based predictive maintenance at smart ports using iot sensor data. *Sensors* **25**(13), 3923 (2025)
- [4] Teoh, Y.K., Gill, S.S., Parlikad, A.K.: Iot and fog-computing-based predictive maintenance model for effective asset management in industry 4.0 using machine learning. *IEEE Internet of Things Journal* **10**(3), 2087–2094 (2021)
- [5] Thomas, J., Patidar, P., Vedi, K.V., Gupta, S.: An analysis of predictive maintenance strategies in supply chain management. *Int J Sci Res Arch* **6**(01), 308–17 (2022)
- [6] Somu, N., Dasappa, N.S.: An edge-cloud iiot framework for predictive maintenance in manufacturing systems. *Advanced Engineering Informatics* **65**, 103388 (2025)
- [7] Kovaleva, Y., Iliashenko, O., Iliashenko, V., Anselm, R.: Implementation of predictive maintenance in a railway logistics company. In: Proceedings of the International Scientific Conference-Digital Transformation on Manufacturing, Infrastructure and Service, pp. 1–7 (2020)
- [8] Atakari, C.: Ai-driven predictive maintenance models in erp systems for critical infrastructure and national defense logistics. *International Journal of Emerging Research in Engineering and Technology* **6**(1), 82–90 (2025)
- [9] Patrício, L., Varela, L., Silveira, Z.: Proposal for a sustainable model for integrating robotic process automation and machine learning in failure prediction and operational efficiency in predictive maintenance. *Applied Sciences* **15**(2), 854 (2025)
- [10] Celestin, M.: How predictive maintenance in logistics fleets is reducing equipment downtime and operational losses. *Brainae Journal of Business, Sciences and Technology (BJBST)* **7**(10), 1023–1033 (2023)
- [11] Patel, P., Patel, R.: Enhancing manufacturing with ai, iot, and machine learning: A focus on predictive maintenance. *J Adv Robot Auton Syst Hum Mach Interact*

1(1), 01–03 (2025)

- [12] Hasan, M.N.: Predictive maintenance optimization for smart vending machines using iot and machine learning. arXiv preprint arXiv:2507.02934 (2025)
- [13] Chen, J., Lim, C.P., Tan, K.H., Govindan, K., Kumar, A.: Artificial intelligence-based human-centric decision support framework: an application to predictive maintenance in asset management under pandemic environments. *Annals of Operations Research* **350**(2), 493–516 (2025)
- [14] Tsallis, C., Papageorgas, P., Piromalis, D., Munteanu, R.A.: Application-wise review of machine learning-based predictive maintenance: Trends, challenges, and future directions. *Applied Sciences* **15**(9), 4898 (2025)
- [15] Voronov, S.: Machine learning models for predictive maintenance. Linkopings Universitet (Sweden) ProQuest Dissertations & Theses (2020)
- [16] Guidotti, D., Pandolfo, L., Pulina, L.: A systematic literature review of supervised machine learning techniques for predictive maintenance in industry 4.0. *IEEE Access* (2025)
- [17] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.-Y.: Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems* **30** (2017)
- [18] Watanabe, S.: Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance. arXiv preprint arXiv:2304.11127 (2023)
- [19] Joy, J., Selvan, M.P.: A comprehensive study on the performance of different multi-class classification algorithms and hyperparameter tuning techniques using optuna. In: 2022 International Conference on Computing, Communication, Security and Intelligent Systems (IC3SIS), pp. 1–5 (2022). IEEE
- [20] Niculescu-Mizil, A., Caruana, R.: Predicting good probabilities with supervised learning. In: *Proceedings of the 22nd International Conference on Machine Learning*, pp. 625–632 (2005)
- [21] Blagec, K., Dorffner, G., Moradi, M., Samwald, M.: A critical analysis of metrics used for measuring progress in artificial intelligence. arXiv preprint arXiv:2008.02577 (2020)
- [22] Lundberg, S.M., Lee, S.-I.: A unified approach to interpreting model predictions. *Advances in neural information processing systems* **30** (2017)
- [23] Ishwaran, H., Kogalur, U.B., Blackstone, E.H., Lauer, M.S.: *Random survival forests* (2008)

- [24] Harrell, F.E., Califf, R.M., Pryor, D.B., Lee, K.L., Rosati, R.A.: Evaluating the yield of medical tests. *Jama* **247**(18), 2543–2546 (1982)

ARTICLE IN PRESS